

MÓDULO 01

Introducción a los AI Agents

Fundamentos para construir el "AI Software Engineer Agent" capaz de asistir en el desarrollo de componentes Lit sobre la arquitectura Cells.

CURSO

Building AI Agents for SWE — Lit & Cells

DURACIÓN MÓDULO NIVEL

2.5 horas

Ingeniería de Software

DE UN VISTAZO

Objetivos del módulo

Antes de escribir una sola línea de código de agentes, necesitamos un lenguaje común: qué es (y qué no es) un AI Agent, en qué se diferencia de un asistente conversacional, y cómo razona para llegar de un requerimiento a un componente Lit terminado.

Al finalizar este módulo serás capaz de:

- **Definir con precisión** qué es un AI Agent y distinguirlo de un simple asistente o chatbot.
- **Comparar** AI Assistant vs AI Agent con criterios objetivos (autonomía, memoria, herramientas, objetivo).
- **Clasificar** los tipos de agentes existentes y ubicar cuál es el más adecuado para automatizar tareas de desarrollo Lit/Cells.
- **Explicar el ciclo de razonamiento** Reason → Plan → Act y trazarlo sobre un caso real de generación de componentes.
- **Identificar casos de uso** concretos de IA aplicada a Software Engineering dentro del stack Lit + Cells de BBVA.

Agenda

01 ¿Qué es un AI Agent?

Definición, componentes y por qué importa para ingeniería de software

02 AI Assistant vs AI Agent

Diferencias clave y tabla comparativa aplicada a Lit

03 Tipos de agentes

Reactivos, deliberativos, basados en objetivos, de aprendizaje y multi-agente

04 Ciclo de razonamiento

Reason → Plan → Act, paso a paso, con ejemplo de un componente customer-card

05 Casos de uso en Software Engineering

Dónde aplica un agente en el ciclo de vida de un componente Cells

06 Ejercicio práctico

Mapeo del proceso actual de creación de un componente Cells

1 ¿Qué es un AI Agent?

Un **AI Agent** es un sistema construido alrededor de un modelo de lenguaje (LLM) que **percibe un entorno, decide una serie de acciones y las ejecuta usando herramientas**, de forma autónoma y hacia un objetivo, ajustando su plan según los resultados que va observando.

La diferencia frente al software tradicional no está en el lenguaje de programación, sino en **quién decide el siguiente paso**: en software tradicional lo decide el desarrollador al escribir el código; en un agente, lo decide el modelo en tiempo de ejecución, dentro de límites que nosotros definimos.

IDEA CLAVE

Software tradicional = reglas fijas escritas por un humano. AI Agent = objetivo fijado por un humano + plan que decide el modelo. El ingeniero deja de programar "el cómo" paso a paso y pasa a diseñar "el objetivo, las herramientas y los límites".

Los 4 componentes de todo agente

🧠 Modelo (LLM)

El "cerebro" que interpreta la instrucción, razona y decide qué hacer. En nuestro curso: GPT-4 / Azure OpenAI.

📁 Memoria

Contexto de corto plazo (la conversación actual) y de largo plazo (convenciones de Cells, estilos, decisiones previas).

🔧 Herramientas (Tools)

Funciones reales que el agente puede invocar: leer un archivo, correr linter, crear un componente, abrir un PR.

🎯 Objetivo / Instrucciones

La meta y las reglas del juego: "genera un componente Lit siguiendo la convención Cells", con sus restricciones.

Ejemplo aplicado

Un desarrollador escribe: *"Necesito un componente que muestre el saldo del cliente"*. El agente:

- Interpreta la intención (mostrar un dato financiero, probablemente un **Cell**).
- Consulta memoria/herramientas: convenciones de naming, componentes similares ya existentes.
- Genera `customer-balance.js`, sus estilos, pruebas y README.
- Verifica el resultado (lint, tests) antes de entregarlo.

2 AI Assistant vs AI Agent

Es la confusión más común al iniciar con IA aplicada a desarrollo. Un **AI Assistant** (como usar Copilot o ChatGPT para autocompletar código) responde a una petición puntual. Un **AI Agent** persigue un objetivo de principio a fin, decidiendo sus propios pasos intermedios y usando herramientas sin que el humano deba pedir cada acción una por una.

Criterio	AI Assistant	AI Agent
Autonomía	Responde una petición a la vez. El humano decide el siguiente paso.	Ejecuta una secuencia de pasos hacia un objetivo sin supervisión continua.
Memoria	Generalmente limitada a la conversación actual.	Mantiene contexto entre pasos y puede persistir convenciones/decisiones.
Uso de herramientas	Sugiere código; el desarrollador copia, pega y ejecuta.	Invoca herramientas directamente: crea archivos, corre tests, abre PRs.
Ejemplo Lit/Cells	"Autocompleta esta propiedad reactiva en mi componente."	"Genera el componente customer-card completo con tests y README."
Rol del ingeniero	Guía cada micro-paso.	Define objetivo, reglas y revisa el resultado final.

PARA RECORDAR

Todo AI Agent usa un LLM como los AI Assistants — la diferencia no es el modelo, es la **arquitectura alrededor del modelo**: bucle de decisión, memoria y herramientas conectadas.

¿Por qué esta distinción importa en BBVA?

El objetivo del curso es construir un **AI Cells Software Engineer Agent**. Eso implica pasar de "pedirle código a un asistente" a "delegarle una historia de usuario completa a un agente" que entrega un componente listo para revisión: código, estilos, pruebas, documentación y Storybook.

3 Tipos de agentes

No todos los agentes razonan igual. Conocer el espectro ayuda a elegir el diseño correcto para cada tarea de ingeniería — desde algo simple (linting) hasta algo complejo (un pipeline multi-agente de desarrollo).

1 · Reactivos (Reactive)

Responden a un estímulo con una acción fija, sin planificación ni memoria. **Ejemplo:** un linter automático que corrige comillas al guardar.

2 · Deliberativos (Planning)

Construyen un plan de varios pasos antes de actuar. **Ejemplo:** un agente que diseña la estructura de un componente antes de generar código.

3 · Basados en objetivos (Goal-based)

Evalúan distintas rutas y eligen la que mejor cumple el objetivo. **Ejemplo:** elegir entre reutilizar un componente existente o crear uno nuevo.

4 · De aprendizaje (Learning)

Ajustan su comportamiento con la retroalimentación recibida. **Ejemplo:** un agente de code review que refina sus reglas según lo que el equipo acepta o rechaza.

5 · Multi-agente (Multi-agent systems)

Varios agentes especializados colaboran en cadena, cada uno responsable de una parte del flujo. Es el modelo que usaremos en el Módulo 13:

Developer Agent

Architect Agent

Reviewer Agent

Testing Agent

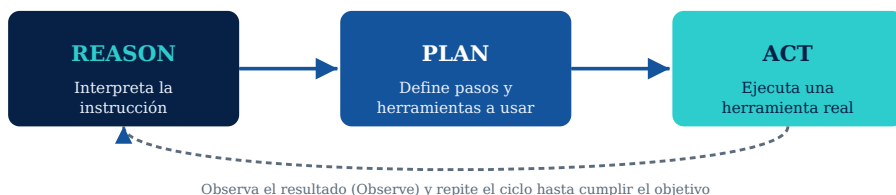
Documentation Agent

¿Cuál usaremos en este curso?

Empezaremos con un agente **deliberativo con herramientas** (Módulo 5) y evolucionaremos hacia un **sistema multi-agente** (Módulo 13) a medida que las tareas se vuelvan más complejas: generar, revisar, probar y documentar un componente Cells de punta a punta.

4 Ciclo de razonamiento: Reason → Plan → Act

Este es el bucle que ejecuta cualquier agente de forma repetida hasta cumplir el objetivo o alcanzar un límite de seguridad (máximo de pasos, tiempo o costo).



Ejemplo trazado — generar el componente customer-card

```
// Instrucción del ingeniero
"Crea un componente Cells que muestre nombre, número de cuenta y saldo del cliente"

// 1) REASON — el agente interpreta
→ Es un componente de presentación de datos de cliente.
→ Debe seguir arquitectura Cells (naming, eventos, i18n, accesibilidad).

// 2) PLAN — decide la secuencia de pasos
1. Revisar componentes similares existentes (evitar duplicar)
2. Definir propiedades reactivas: name, accountNumber, balance
3. Generar customer-card.js, customer-card.scss, test y README

// 3) ACT — invoca herramientas reales
tool.createFile("customer-card.js", ...)
tool.createFile("customer-card.test.js", ...)
tool.runLinter("customer-card.js")
```

Este mismo bucle —Reason, Plan, Act, y Observe— es el que veremos con más detalle y código funcional en el Módulo 2 (Arquitectura de un AI Agent) y lo construiremos por primera vez en el Módulo 5.

5 Casos de uso en Software Engineering

Un agente puede insertarse en casi cualquier punto del ciclo de vida de un componente Lit/Cells. Estos son los casos de uso que desarrollaremos a lo largo del curso — cada uno corresponde a un módulo posterior:

Caso de uso	Qué automatiza	Módulo
Generación de componentes	Convierte una historia de usuario en un componente Lit siguiendo convenciones Cells.	5, 6, 7
Code Review asistido	Revisa Clean Code, accesibilidad, buenas prácticas de Lit, performance y seguridad.	8
Refactoring	Detecta malas prácticas en componentes antiguos y los migra/optimiza.	9
Generación de pruebas	Crea unit tests, mocks y fixtures con @open-wc/testing.	10
Documentación automática	Genera README con props, eventos, slots, ejemplos e instalación.	11
Storybook automático	Crea historias, controles y variantes del componente.	12
Integración con GitHub / Azure DevOps	Lee issues o user stories y abre Pull Requests con el trabajo generado.	14, 15

POR QUÉ IMPORTA PARA BBVA

Estandarizar estos casos de uso con agentes reduce tiempo de desarrollo y, al mismo tiempo, **refuerza la consistencia** de la librería de componentes Cells: naming, accesibilidad e internacionalización dejan de depender de la memoria de cada desarrollador.

EJERCICIO PRÁCTICO · 30 MIN

Mapea tu proceso actual

Analiza el proceso actual para crear un componente Cells e identifica qué actividades pueden automatizarse con IA.

Instrucciones

- En parejas o de forma individual, escribe los **pasos reales** que sigues hoy para crear un componente Cells nuevo (desde que recibes el requerimiento hasta el Pull Request aprobado).
- Para cada paso, clasifícalo con una de las etiquetas: Manual Automatizable hoy
Requiere agente .
- Identifica cuál de los 5 tipos de agentes (sección 3) aplicaría mejor a cada paso marcado como "Requiere agente".
- Señala en qué punto del ciclo Reason → Plan → Act encajaría cada automatización propuesta.

Plantilla sugerida

#	Paso del proceso actual	Clasificación	Tipo de agente / notas
1	Recibir historia de usuario		
2	Definir props y eventos		
3	Escribir el componente		
4	Escribir pruebas unitarias		
5	Documentar (README / Storybook)		
6	Code review		

ENTREGABLE

Tabla completa + 3 conclusiones sobre dónde un AI Agent generaría más impacto en tu equipo. Se retoma en el Módulo 2 para diseñar el diagrama del primer agente.

✓ Resumen del módulo

Qué es un AI Agent

LLM + memoria + herramientas + objetivo, actuando de forma autónoma hacia una meta.

Assistant vs Agent

El asistente responde una petición; el agente ejecuta una secuencia completa sin supervisión paso a paso.

Tipos de agentes

Reactivos, deliberativos, basados en objetivos, de aprendizaje y multi-agente.

Reason → Plan → Act

El bucle de razonamiento que repite todo agente hasta cumplir el objetivo.

Próximo módulo

MÓDULO 2 · ARQUITECTURA DE UN AI AGENT

Profundizaremos en cada componente del agente — LLM, Memory, Tools, Planning, Reflection, Multi-step reasoning y Function Calling— y diseñaremos el diagrama del agente que generará componentes Lit.

MÓDULOS 02 — 17 + PROYECTO FINAL

Building AI Agents for Software Engineers

Continuación del programa: de la arquitectura de un agente a un sistema multi-agente conectado a GitHub, Azure DevOps y MCP, capaz de generar componentes Lit bajo la arquitectura Cells.

CURSO

Building AI Agents for SWE — Lit & Cells

DURACIÓN TOTALPROYECTO FINAL

40 horas**AI Cells Software Engineer**

02

MÓDULO 02

Arquitectura de un AI Agent

Abrimos la caja: cómo se conectan LLM, memoria, herramientas, planificación y reflexión para que un agente pase de una instrucción a un componente Lit terminado.

→ OBJETIVOS

- Explicar el rol de cada bloque: LLM, Memory, Tools, Planning, Reflection.
- Distinguir razonamiento de un solo paso vs. multi-step reasoning.
- Entender Function Calling como el puente entre el modelo y el código real.
- Diseñar el diagrama de un agente generador de componentes Lit.

1 Los 7 bloques de un agente

En el Módulo 1 vimos 4 componentes a alto nivel. Ahora los desagregamos en los 7 bloques que sí necesitas conocer para **implementarlos en código**.

LLM	El motor de razonamiento (GPT-4, Azure OpenAI). Recibe el prompt del sistema + el contexto y devuelve texto o una llamada a función.
Memory	Corto plazo: la conversación/tarea actual. Largo plazo: convenciones de Cells, decisiones previas, componentes ya generados (Módulo 17).
Tools	Funciones reales expuestas al modelo: <code>createFile()</code> , <code>runLinter()</code> , <code>readComponent()</code> , <code>openPullRequest()</code> .
Planning	Descompone el objetivo en subtarefas ordenadas antes de actuar (p. ej. "props → template → estilos → tests").
Reflection	El agente evalúa su propio resultado (¿pasa el linter? ¿falta un evento?) y decide si debe corregir antes de entregar.
Multi-step reasoning	Encadena varias llamadas al LLM, usando la salida de un paso como entrada del siguiente.
Function Calling	El mecanismo formal por el cual el LLM "pide" ejecutar una tool con argumentos estructurados (JSON), en vez de solo responder texto.

Function Calling en la práctica

```
// El LLM no ejecuta código: devuelve la INTENCIÓN de llamar a una función
{
  "tool_call": {
    "name": "createLitComponent",
    "arguments": {
      "name": "customer-card",
      "properties": ["name", "accountNumber", "balance"],
      "events": ["card-selected"]
    }
  }
}

// Nuestro código (el "runtime" del agente) ejecuta la función real
const result = await tools.createLitComponent(args);
```

IDEA CLAVE

El LLM decide **qué** hacer y con **qué argumentos**; el runtime del agente (nuestro código) decide **cómo** ejecutarlo de forma segura. Esa separación es la que hace posible auditar y limitar lo que un agente puede tocar.

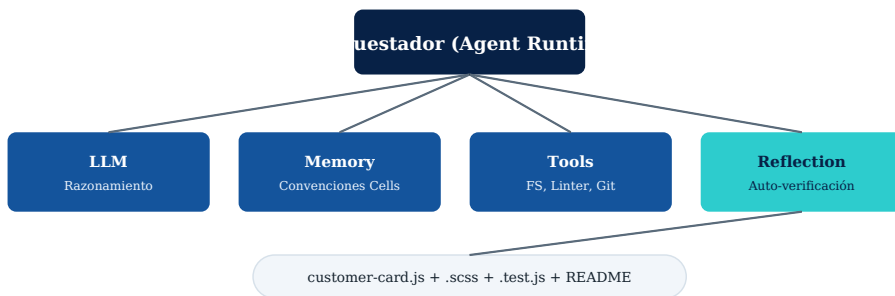
2 Multi-step reasoning en acción

Generar un componente rara vez es una sola llamada al LLM. Así se ve el flujo encadenado para customer-card:



Cada caja es una llamada independiente al LLM y/o a una tool; el resultado de una alimenta a la siguiente.

Arquitectura de referencia



EJERCICIO · DISEÑO DEL DIAGRAMA DEL AGENTE

Diseña el diagrama de un agente que pueda generar componentes Lit para BBVA. Debe incluir explícitamente:

- Qué Tools necesita (mínimo 3: crear archivo, leer convenciones, correr linter).
- Qué guarda en Memory de largo plazo (convenciones, historial de componentes).
- En qué paso ocurre la Reflection y qué pasa si falla la validación.

03

MÓDULO 03

Lit + Cells Architecture

Antes de construir agentes que generen componentes, necesitamos dominar el framework que esos agentes van a escribir por nosotros: Lit por dentro, y la arquitectura Cells por encima.

→ OBJETIVOS

- Dominar los fundamentos de Lit: Shadow DOM, propiedades reactivas, decoradores, lifecycle, slots, templates y directivas.
- Entender la arquitectura Cells: Components, Pages, Views, Routing, Services, Events, State.
- Construir manualmente el componente customer-card, para luego comparar contra la versión generada por IA.

1 Fundamentos de Lit

Shadow DOM	Encapsula DOM y CSS del componente; nada se filtra hacia afuera ni desde afuera (aislamiento de estilos).
Reactive Properties	Propiedades que, al cambiar, disparan un re-render automático del componente.
Decorators	@property, @state, @query — sintaxis declarativa sobre la clase del componente.
Lifecycle	connectedCallback, updated, firstUpdated — ganchos para efectos secundarios.
Slots	Puntos de proyección de contenido HTML externo dentro del Shadow DOM del componente.
Templates	Tagged template literals (html`...`) — Lit sólo re-renderiza lo que cambió.
Directives	Funciones especiales dentro de un template (classMap, repeat, ifDefined) que optimizan el render.

Anatomía mínima de un componente Lit

```
import { LitElement, html, css } from 'lit';
import { customElement, property } from 'lit/decorators.js';

@customElement('customer-card')
export class CustomerCard extends LitElement {
  @property({ type: String }) name = '';
  @property({ type: Number }) balance = 0;

  static styles = css`
    :host { display: block; border-radius: 8px; padding: 16px; }
  `;

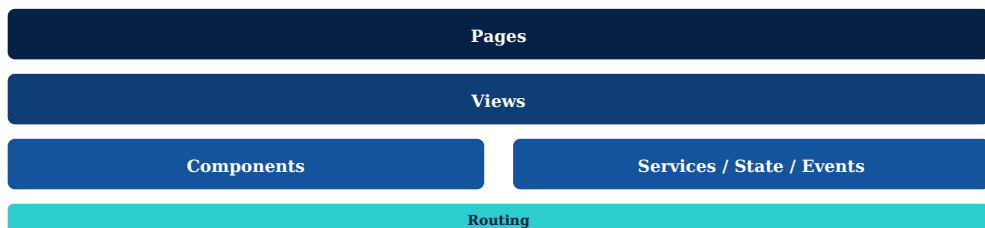
  render() {
    return html`
      <h3>${this.name}</h3>
      <p>Saldo: ${this.balance}</p>
      <slot name="footer"></slot>
    `;
  }
}
```

POR QUÉ LE IMPORTA A UN AGENTE

Un agente que genera Lit necesita "saber" estas reglas de memoria (Módulo 17) para no producir componentes que rompan el encapsulamiento, dupliquen estilos globales o ignoren el lifecycle correcto.

2 Arquitectura Cells

Cells organiza los componentes Lit de BBVA en capas con responsabilidades claras. Un agente debe respetar esta jerarquía al decidir dónde y cómo generar código.



Components	Piezas reutilizables y "tontas" (presentacionales), como customer-card.
Pages	Pantallas completas, orquestan Views y conectan con Routing.
Views	Secciones dentro de una Page que agrupan varios Components con lógica propia.
Routing	Define qué Page se renderiza según la URL/estado de navegación.
Services	Acceso a datos/API, independientes de la UI (p. ej. <code>customerService.getBalance()</code>).
Events	Comunicación entre componentes vía Custom Events, nunca acoplamiento directo.
State	Estado compartido entre componentes/Views (store ligero o Context API de Lit).

LABORATORIO · CREAR CUSTOMER-CARD

Construye manualmente el componente customer-card con:

- **Propiedades:** name, accountNumber, balance.
- **Eventos:** emite card-selected al hacer click.
- **Slots:** un slot footer para acciones contextuales.
- **Estilos:** encapsulados con `static styles`, siguiendo tokens de color de BBVA.

Guarda este componente — en el Módulo 5 el agente generará su propia versión y las compararemos.

04

MÓDULO 04

Prompt Engineering para Lit

La calidad del código que genera un agente depende directamente de la calidad del prompt. Aprenderemos a escribir prompts que produzcan componentes, estilos, tests, Storybook y documentación consistentes.

→ OBJETIVOS

- Aplicar una estructura de prompt reutilizable para generación de código Lit.
- Usar ejemplos (few-shot) para fijar convenciones de BBVA/Cells.
- Construir prompts reutilizables para bbva-card, bbva-button y bbva-modal.

1 Anatomía de un buen prompt de generación

Un prompt para generar código de producción necesita mucho más contexto que uno conversacional. Usamos 5 bloques fijos:

Rol	"Eres un ingeniero senior de Frontend en BBVA, experto en Lit y en la arquitectura Cells."
Contexto	Convenciones de naming, stack (Lit 3, TypeScript), estructura de carpetas esperada.
Tarea	Qué debe generar exactamente: componente + estilos + tests + README.
Restricciones	Accesibilidad (WCAG AA), i18n obligatorio, sin librerías externas no aprobadas.
Formato de salida	Uno o varios bloques de código con el nombre de archivo indicado explícitamente.

Ejemplo de prompt reutilizable — generar un Component

```
// System prompt (fijo, memoria de largo plazo)
Eres un ingeniero senior de Frontend en BBVA. Generas componentes
Lit siguiendo la arquitectura Cells: naming en kebab-case con
prefijo del dominio, propiedades tipadas, eventos documentados,
soporte i18n y accesibilidad WCAG AA. Nunca uses librerías externas
no aprobadas. Responde solo con bloques de código, un archivo por bloque.

// User prompt (varía por tarea)
Genera el componente "bbva-card" con:
- properties: title, subtitle, imageUrl
- evento "card-click"
- slot "actions"
Entrega: bbva-card.js, bbva-card.scss, bbva-card.test.js, README.md
```

BUENAS PRÁCTICAS

Fija el *system prompt* como memoria de largo plazo (Módulo 17) y varía solo el *user prompt* por tarea — así todos los componentes generados comparten las mismas convenciones sin repetirlas cada vez.

2 Prompts por tipo de artefacto

Components

Especifica properties, eventos, slots y estilos esperados. Pide siempre TypeScript + decoradores.

Styles

Referencia los design tokens de BBVA (colores, espaciados) en vez de valores hardcodeados.

Tests

Pide explícitamente @open-wc/testing, cobertura de props, eventos y estados de error.

Storybook + Documentación

Pide controles por cada property y al menos 3 variantes (default, con error, deshabilitado).

EJERCICIO · PROMPTS REUTILIZABLES

Escribe 3 prompts reutilizables (siguiendo la estructura de 5 bloques) para generar:

- bbva-card — tarjeta de contenido con imagen, título y acción.
- bbva-button — botón con variantes primary/secondary/disabled.
- bbva-modal — modal con slot de contenido y evento de cierre.

Guarda estos 3 prompts — los usaremos tal cual en el Módulo 6 para probarlos contra el agente real.

05

MÓDULO 05

Construcción del primer AI Agent

De la teoría a un agente funcionando: elegimos herramientas (OpenAI, Azure OpenAI, LangChain, Semantic Kernel) y construimos el primer agente end-to-end.

→ OBJETIVOS

- Comparar OpenAI SDK, Azure OpenAI, LangChain y Semantic Kernel para este caso de uso.
- Implementar el bucle Reason-Plan-Act con Function Calling real.
- Generar customer-profile.js, customer-profile.css y README.md con el primer agente.

1 Herramientas para construir agentes

Herramienta	Cuándo usarla	Nota para BBVA
OpenAI SDK	Control total y mínima abstracción; ideal para aprender el bucle Reason-Plan-Act desde cero.	Usaremos esta opción en el laboratorio de este módulo.
Azure OpenAI	Mismo modelo, con los controles de seguridad, red y compliance que exige la infraestructura del banco.	Opción recomendada para llevar el agente a un entorno productivo.
LangChain	Cuando se necesitan cadenas complejas, múltiples tools y integraciones ya construidas.	Útil a partir del Módulo 13 (sistema multi-agente).
Semantic Kernel	Entornos .NET / C#, con "plugins" tipados equivalentes a las tools.	Alternativa si el equipo backend ya trabaja en .NET.

El primer agente — estructura mínima

```
const tools = [
  { name: 'createFile', description: 'Crea un archivo con contenido' },
  { name: 'readConventions', description: 'Lee convenciones Cells' }
];

async function runAgent(userRequest) {
  let messages = [
    { role: 'system', content: SYSTEM_PROMPT },
    { role: 'user', content: userRequest }
  ];
  while (true) {
    const response = await llm.chat({ messages, tools });
    if (response.tool_call) {
      const result = await executeTool(response.tool_call);
      messages.push({ role: 'tool', content: result });
    } else {
      return response.content; // tarea completada
    }
  }
}
```

IDEA CLAVE

Este bucle while es la implementación literal de Reason → Plan → Act visto en el Módulo 1: cada vuelta el modelo decide si necesita otra herramienta o si ya puede responder.

2 Caso guiado: customer-profile

El agente recibirá la instrucción "**Crear un componente customer-profile**" y deberá producir los 3 archivos esperados.

Entrada

"Crear un componente customer-profile" (sin más detalle — el agente debe inferir props razonables a partir de las convenciones en memoria).

Salida esperada

customer-profile.js, customer-profile.css, README.md.

LABORATORIO · CONSTRUIR EL PRIMER AGENTE

- Configura el cliente del LLM (OpenAI o Azure OpenAI) con tu API key de práctica.
- Implementa las tools `createFile` y `readConventions`.
- Ejecuta el agente con la instrucción de `customer-profile` y revisa los 3 archivos generados.
- Compara el resultado contra el `customer-card` que construiste a mano en el Módulo 3: ¿siguió las mismas convenciones?

CHECKLIST DE ACEPTACIÓN

✓ Archivos con el nombre correcto · ✓ Propiedades reactivas coherentes · ✓ CSS sin estilos globales filtrados · ✓ README con al menos props y un ejemplo de uso.

06

MÓDULO 06

AI Agent para generar componentes Lit

Aquí el agente deja de ser un experimento: recibe una descripción en lenguaje natural y entrega un componente Lit completo, listo con tests, documentación y Storybook.

→ OBJETIVOS

- Extender el agente del Módulo 5 para generar props, eventos, documentación y ejemplos.
- Probar el agente con un caso realista: un Login Form.
- Agregar validaciones automáticas al resultado generado.

1 Caso guiado: Login Form

Input

"Necesito un componente Login Form"

Output

login-form.js · login-form.css · tests ·
README · Storybook

Lo que el agente debe generar en cada archivo

Propiedades	username, password, loading, errorMessage.
Eventos	login-submit con el payload { username, password }.
Documentación	README con tabla de props, eventos y un ejemplo de integración.
Ejemplos	Al menos un ejemplo de uso básico y uno con manejo de error.

```
// Fragmento esperado de login-form.js
@customElement('login-form')
export class LoginForm extends LitElement {
  @property({ type: String }) username = '';
  @property({ type: Boolean }) loading = false;
  @property({ type: String }) errorMessage = '';

  _onSubmit(e) {
    e.preventDefault();
    this.dispatchEvent(new CustomEvent('login-submit', {
      detail: { username: this.username },
      bubbles: true, composed: true
    }));
  }
}
```


2 Validaciones automáticas

Un componente generado no se acepta a ciegas. Agregamos un paso de **Reflection** (Módulo 2) que corre validaciones antes de entregar el resultado al desarrollador.

Validación estructural

¿Existen los 5 archivos esperados? ¿El nombre del custom element coincide con el pedido?

Validación de calidad

¿Pasa ESLint? ¿Las propiedades públicas están tipadas? ¿Hay al menos un test por evento?

EJERCICIO · AGREGAR VALIDACIONES AUTOMÁTICAS

- Añade una tool `validateComponent(files)` que el agente ejecute automáticamente tras generar el código.
- Si la validación falla, el agente debe volver al paso de generación con el detalle del error (ciclo de Reflection).
- Prueba el flujo completo con el prompt de `login-form` forzando un error (por ejemplo, quitar un evento) y confirma que el agente lo detecta.

07

MÓDULO 07

Agente para generar componentes Cells

El agente ahora conoce las convenciones específicas de Cells: naming, arquitectura, eventos, internacionalización y accesibilidad, y las aplica de forma consistente.

→ OBJETIVOS

- Extender el agente para producir la estructura completa de un paquete Cells.
- Incorporar Naming Convention, i18n y Accesibilidad como reglas de generación, no como revisión posterior.
- Construir un agente especializado en Cells de punta a punta.

1 Caso guiado: cells-login

A diferencia del Módulo 6, aquí el agente debe generar el **paquete completo** siguiendo la estructura de un módulo Cells publicable.

cells-login.js	Componente principal, extiende de LitElement bajo convención Cells.
cells-login.scss	Estilos con tokens de diseño BBVA (no CSS-in-JS suelto).
cells-login.test.js	Pruebas unitarias base del componente.
demo/	Ejemplo ejecutable del componente en aislamiento.
README.md	Documentación de props, eventos y uso.
package.json	Metadatos del paquete, listo para publicarse en el registro interno.

Reglas que el agente debe seguir siempre

Naming Convention

Prefijo `cells-`, kebab-case, sin abreviaturas ambiguas.

Internacionalización

Ningún texto hardcodeado; todo pasa por el servicio de i18n del proyecto.

2 Accesibilidad como regla de generación

La diferencia entre "generar código que funciona" y "generar código listo para Cells" está en tratar la accesibilidad como un requisito de entrada, no como una corrección posterior.

```
// El agente debe incluir esto por default, sin que se lo pidan
render() {
  return html`
    <label for="username">${this.i18n.t('login.username')}</label>
    <input id="username" aria-required="true" .value=${this.username} />
  `;
}
```

LABORATORIO · AGENTE ESPECIALIZADO EN CELLS

- Extiende el system prompt del Módulo 5 con las 5 reglas de Cells (naming, arquitectura, eventos, i18n, accesibilidad).
- Agrega una tool `scaffoldCellsPackage(name)` que genere la estructura de carpetas y archivos completa.
- Genera `cells-login` de punta a punta y verifica cada archivo contra la lista de la página anterior.

08

MÓDULO 08

AI Agent para Code Review

El agente recibe un Pull Request y devuelve comentarios accionables sobre Clean Code, accesibilidad, buenas prácticas de Lit, performance y seguridad.

→ OBJETIVOS

- Diseñar el prompt de revisión con criterios objetivos y verificables.
- Generar comentarios en el formato que el equipo usa hoy en sus PRs.
- Revisar un componente existente con el agente.

1 Los 5 ejes de revisión

Clean Code	Nombres claros, funciones cortas, sin lógica duplicada.
Accesibilidad	Etiquetas, roles ARIA, contraste, navegación por teclado.
Lit Best Practices	Uso correcto de reactive properties, lifecycle y directivas.
Performance	Evitar renders innecesarios, listas sin repeat/keys, cálculos pesados en render().
Security	Sanitización de HTML dinámico, evitar unsafeHTML sin justificación.

Formato de salida esperado

```
✗ Esta propiedad debería ser reactive.  
✗ No utilices querySelector.  
✓ Usa @query.  
✓ Usa css tagged template.
```

POR QUÉ ESTE FORMATO

Comentarios cortos, con ✗/✓ y una recomendación concreta, son más fáciles de accionar en un PR que un párrafo largo — y es el mismo formato que ya usan los revisores humanos en BBVA.

2 Diseño del agente revisor

```
// Prompt de sistema del Reviewer Agent
Revisa el siguiente componente Lit contra 5 ejes: Clean Code,
Accesibilidad, Lit Best Practices, Performance y Security.
Para cada hallazgo, responde con una línea iniciando en ✖ (problema)
o ✔ (recomendación), sin párrafos largos. No reescribas el archivo
completo, solo comenta.
```

EJERCICIO · REVISAR UN COMPONENTE EXISTENTE

- Toma el customer-card del Módulo 3 (o el generado en el Módulo 5) y pásalo por el Reviewer Agent.
- Clasifica cada comentario recibido según los 5 ejes.
- Discute en grupo: ¿hubo algún falso positivo? ¿qué tan útil fue comparado con una revisión humana?

09

MÓDULO 09

Agente para Refactoring

El agente recibe un componente antiguo, detecta malas prácticas y lo migra, optimiza y renombra sin cambiar su comportamiento observable.

→ OBJETIVOS

- Diferenciar refactoring de reescritura: preservar comportamiento.
- Detectar patrones obsoletos comunes en componentes Lit heredados.
- Refactorizar un componente Cells antiguo real.

1 Qué debe detectar el agente

Malas prácticas

Manipulación directa del DOM (`querySelector`), propiedades sin tipar, CSS global filtrado.

Migración

Decoradores antiguos o sintaxis Lit 1/2 a la sintaxis actual (Lit 3 + TypeScript).

Optimizar render

Evitar recalcular listas o estilos en cada `render()`; mover a `willUpdate`.

Mejorar nombres

Renombrar propiedades y métodos ambiguos siguiendo el naming convention de Cells.

```
// Antes
updateBalance(v) { this.shadowRoot.querySelector('#balance').innerText = v; }

// Después — el agente lo convierte en propiedad reactiva
@property({ type: Number }) balance = 0;
// El template ya se re-renderiza solo al cambiar "balance"
```

2 Garantizar que nada se rompió

Todo refactor generado por IA debe pasar por una validación de equivalencia antes de aceptarse.

REGLA DE ORO

Si el componente tenía tests antes del refactor, deben seguir pasando **sin modificarlos** después. Si no tenía tests, el Módulo 10 nos dará el agente para generarlos primero.

LABORATORIO · REFACTORIZAR UN COMPONENTE CELLS ANTIGUO

- Toma (o simula) un componente Lit con al menos 2 malas prácticas de la lista anterior.
- Pide al agente un plan de refactor antes de tocar código (paso de Planning, Módulo 2).
- Aplica el refactor y corre los tests existentes — deben seguir en verde.

10

MÓDULO 10

Agente para generar pruebas

El agente crea automáticamente unit tests, mocks y fixtures usando @open-wc/testing o Web Test Runner, cubriendo props, eventos y estados de error.

→ OBJETIVOS

- Definir qué debe cubrir un test unitario de un componente Lit.
- Generar mocks y fixtures reutilizables.
- Generar pruebas para customer-card.

1 Qué debe cubrir la suite generada

Unit Tests	Render inicial, cambio de cada propiedad reactiva, emisión de cada evento público.
Mock	Servicios externos (p. ej. <code>customerService</code>) simulados, sin llamadas reales.
Fixtures	Datos de ejemplo reutilizables entre distintos archivos de test.

```
// Ejemplo generado con @open-wc/testing
import { fixture, html, expect } from '@open-wc/testing';
import '../customer-card.js';

describe('customer-card', () => {
  it('renderiza el nombre del cliente', async () => {
    const el = await fixture(html`<customer-card name="Ana"></customer-card>\`);
    expect(el.shadowRoot.textContent).to.include('Ana');
  });

  it('emite card-selected al hacer click', async () => {
    const el = await fixture(html`<customer-card></customer-card>\`);
    setTimeout(() => el.click());
    const { detail } = await oneEvent(el, 'card-selected');
    expect(detail).to.exist;
  });
});
```

2 Cobertura como criterio de aceptación

No basta con que existan tests: el agente debe reportar qué props y eventos cubrió, para que el desarrollador identifique huecos.

CHECKLIST MÍNIMO POR COMPONENTE

✓ Un test por propiedad pública · ✓ Un test por evento emitido · ✓ Al menos un test de estado de error o vacío.

EJERCICIO · GENERAR PRUEBAS DE CUSTOMER-CARD

- Ejecuta el agente de este módulo sobre el customer-card ya construido.
- Verifica que cubra name, accountNumber, balance y el evento card-selected.
- Corre la suite generada con Web Test Runner y confirma que pasa en verde.

11

MÓDULO 11

Agente para documentación

A partir del código fuente de un componente, el agente genera un README completo: propiedades, eventos, slots, ejemplos, instalación y API.

→ OBJETIVOS

- Extraer documentación directamente del código (props, eventos, slots) sin inventar información.
- Generar ejemplos de uso realistas.
- Documentar automáticamente una librería completa de componentes.

1 Caso guiado: customer-card.js → README

Input

customer-card.js (el código fuente real, no una descripción).

Output

README.md con secciones fijas: props, eventos, slots, ejemplos, instalación, API.

```
# customer-card

## Instalación
npm install @bbva/customer-card

## Propiedades
| Prop | Tipo | Descripción |
|-----|-----|-----|
| name | String | Nombre del cliente |
| balance | Number | Saldo disponible |

## Eventos
| Evento | Detalle |
|-----|-----|
| card-selected | { id: string } |

## Ejemplo
<customer-card name="Ana" balance="1200"></customer-card>
```

REGLA ANTI-ALUCINACIÓN

El agente debe documentar **solo lo que existe en el código** — nunca inventar props o eventos que no estén declarados, aunque "suenen razonables".

2 Escalar a una librería completa

Documentar un componente es un caso trivial; documentar una librería significa recorrer múltiples archivos y mantener un índice consistente entre todos los README.



LABORATORIO · DOCUMENTAR TODA UNA LIBRERÍA

- Reúne 3-4 componentes ya generados en módulos anteriores (customer-card, login-form, cells-login...).
- Ejecuta el agente de documentación sobre cada uno.
- Pide al agente un `INDEX.md` final que liste todos los componentes con un link a su README.

12

MÓDULO 12

AI Agent para Storybook

El agente genera el archivo de historias con controles, variantes y ejemplos, para que cualquier componente sea explorable visualmente sin escribir Storybook a mano.

→ OBJETIVOS

- Generar stories.js con controles por cada propiedad pública.
- Cubrir variantes clave: default, error, deshabilitado, loading.

1 De propiedades a controles

El agente traduce cada `@property` del componente en un control de Storybook del tipo adecuado (texto, número, boolean, select).

```
// login-form.stories.js (generado)
export default {
  title: 'Cells/LoginForm',
  component: 'login-form',
  argTypes: {
    loading: { control: 'boolean' },
    errorMessage: { control: 'text' }
  }
};

export const Default = { args: { loading: false } };
export const ConError = { args: { errorMessage: 'Credenciales inválidas' } };
export const Cargando = { args: { loading: true } };
```

IDEA CLAVE

Las variantes no son arbitrarias: deben corresponder a estados reales que el componente puede tener en producción (éxito, error, carga), tomados del propio código.

2 Integración en el pipeline de generación

A partir de aquí, cada componente que el agente genere (Módulos 6-7) debe incluir automáticamente su `stories.js` — dejamos de tratarlo como un paso manual separado.

EJERCICIO

- Genera el `stories.js` para `login-form` y para `cells-login`.
- Verifica que cada variante generada corresponda a un estado real soportado por el componente.
- Abre Storybook localmente y confirma que los controles modifican el componente en vivo.

13

MÓDULO 13

Multi-Agent System

Hasta ahora cada agente hacía una sola tarea. Ahora los encadenamos: Developer, Architect, Reviewer, Testing y Documentation Agent colaborando en un mismo pipeline.

→ OBJETIVOS

- Diseñar un pipeline de agentes especializados con responsabilidades no superpuestas.
- Definir el contrato de datos que pasa de un agente al siguiente.
- Construir un pipeline completo de principio a fin.

1 El pipeline de 5 agentes



Architect Agent Recibe la historia de usuario y decide props, eventos y estructura del componente (sin escribir código).

Developer Agent Toma el diseño del Architect y genera el componente Lit (Módulos 6-7).

Reviewer Agent Aplica los 5 ejes del Módulo 8 sobre el código generado.

Testing Agent Genera la suite de pruebas del Módulo 10 a partir del código ya revisado.

Documentation Agent Genera README y Storybook (Módulos 11-12) como último paso.

2 El contrato entre agentes

Cada agente recibe un objeto estructurado del anterior, no texto libre — esto evita que la información se "pierda en la traducción" entre pasos.

```
// Salida del Architect Agent → entrada del Developer Agent
{
  "componentName": "customer-balance-view",
  "properties": [{ "name": "balance", "type": "Number" }],
  "events": ["balance-refresh"],
  "allyNotes": "Debe anunciarse el saldo a lectores de pantalla"
}
```

IDEA CLAVE

Un sistema multi-agente no es "varios prompts encadenados" — es varios agentes con **responsabilidad única** y un **contrato de datos explícito** entre ellos, igual que microservicios.

LABORATORIO · CONSTRUIR EL PIPELINE COMPLETO

- Define el esquema JSON que pasará entre cada par de agentes.
- Conecta los 5 agentes construidos en módulos anteriores en un único orquestador.
- Ejecuta el pipeline con una historia de usuario simple y verifica que cada agente reciba lo que necesita del anterior.

14

MÓDULO 14

Agente conectado a GitHub

Sacamos al agente del entorno local: ahora puede leer Issues, crear ramas, generar código y abrir Pull Requests directamente sobre el repositorio.

→ OBJETIVOS

- Añadir tools de GitHub (Issues, branches, PRs) al agente del Módulo 13.
- Definir los límites de permisos del agente sobre el repositorio.

1 Nuevas tools: acciones sobre el repositorio

readIssue(id) Obtiene título, descripción y criterios de aceptación de un Issue.

createBranch(name) Crea una rama a partir de main con naming feature/<issue-id>.

commitFiles(files) Aplica el código generado por el pipeline del Módulo 13 en la rama.

openPullRequest() Abre el PR con la descripción generada por el Documentation Agent.

```
// Flujo end-to-end
issue = await tools.readIssue('BBVA-1423');
plan = await architectAgent.run(issue);
code = await developerAgent.run(plan);
review = await reviewerAgent.run(code);
tests = await testingAgent.run(code);
docs = await docAgent.run(code);

await tools.createBranch(`feature/${issue.id}`);
await tools.commitFiles([...code, ...tests, ...docs]);
await tools.openPullRequest();
```

LÍMITE DE SEGURIDAD

El agente **nunca** hace merge directo a main. Su alcance termina en abrir el PR; la aprobación sigue siendo responsabilidad de un ingeniero humano.

EJERCICIO

Conecta el pipeline del Módulo 13 a un repositorio de práctica y ejecútalo de punta a punta contra un Issue de ejemplo, verificando que el PR final contenga código, tests y documentación.

15

MÓDULO 15

Agente conectado a Azure DevOps

Misma idea que GitHub, adaptada al ecosistema que gran parte de los equipos de BBVA ya usa: el agente recibe una User Story de Azure Boards y entrega componente, pruebas y documentación.

→ OBJETIVOS

- Mapear conceptos: Issue de GitHub $\approx$ User Story de Azure Boards.
- Adaptar el pipeline del Módulo 13 a la API de Azure DevOps.

1 De la User Story al entregable

Recibe

Una User Story de Azure Boards: "Como cliente quiero visualizar mi saldo disponible para poder consultar mi cuenta."

Genera

Componente Lit/Cells + pruebas + documentación, igual que en el pipeline de GitHub.

Concepto GitHub	Equivalente Azure DevOps	Tool del agente
Issue	User Story / Work Item	<code>getWorkItem(id)</code>
Branch	Branch (Azure Repos)	<code>createBranch(name)</code>
Pull Request	Pull Request (Azure Repos)	<code>openPullRequest()</code>
Comentario en PR	Comentario en Work Item	<code>commentOnWorkItem()</code>

EJERCICIO

Adapta las tools de GitHub del Módulo 14 a los endpoints REST de Azure DevOps y corre el mismo pipeline contra un Work Item de práctica.

16

MÓDULO 16

AI Agent con MCP (Model Context Protocol)

En vez de codificar cada integración a mano, conectamos el agente a herramientas externas mediante un protocolo estándar: MCP.

→ OBJETIVOS

- Entender qué problema resuelve MCP frente a integraciones ad-hoc.
- Conectar el agente a documentación interna de Cells, estándares de diseño y componentes existentes.
- Construir un agente que consulte documentación antes de generar código.

1 Por qué MCP en vez de tools a medida

Hasta el Módulo 15 cada integración (GitHub, Azure DevOps) se programó a mano. MCP estandariza **cómo un agente descubre y llama herramientas externas**, para no reescribir ese conector cada vez.

Sin MCP

Cada nueva fuente de datos (docs, design system, APIs) requiere código de integración específico y mantenimiento propio.

Con MCP

El agente se conecta a un servidor MCP que expone recursos/herramientas de forma uniforme; agregar una fuente nueva es plug-and-play.

Fuentes que conectaremos

- Documentación interna de Cells (convenciones, arquitectura).
- Estándares de diseño (design tokens, guías de accesibilidad).
- Componentes existentes, para reutilizarlos en vez de duplicarlos.
- Historias de usuario o APIs del sistema de diseño.

2 Consultar antes de generar

La mejora clave frente al Módulo 7 no es técnica sino de secuencia: el agente **consulta** antes de decidir, en vez de generar directamente desde su prompt de sistema.

1. Recibe instrucción



2. Consulta MCP:
docs Cells



3. Consulta MCP:
componentes
existentes



4. Genera solo si no
existe ya

IDEA CLAVE

Esto reduce directamente la duplicación de componentes: si ya existe un bbva-card equivalente, el agente lo reutiliza en vez de crear una variante más.

EJERCICIO

Conecta el agente a un servidor MCP (o simulado) con documentación de componentes existentes, y verifica que antes de generar bbva-card consulte si ya existe algo equivalente.

17

MÓDULO 17

Agente con memoria

El último bloque antes del proyecto final: el agente recuerda convenciones, arquitectura y estilos entre sesiones, para que no haya que repetirlas en cada instrucción.

→ OBJETIVOS

- Diferenciar memoria de corto y largo plazo (Módulo 2) e implementar la de largo plazo.
- Persistir reglas como "usa siempre componentes BBVA" sin repetirlas por prompt.

1 Qué debe recordar el agente

Convenciones	Naming, estructura de carpetas, prefijos de componente (Módulo 7).
Arquitectura	Reglas de Cells: dónde va un Component vs. una Page vs. un Service.
Estilos	Tokens de diseño BBVA y preferencias fijadas por el equipo.

```
// Antes: había que repetirlo en cada prompt
"Genera un bbva-card... usa siempre componentes BBVA, sigue Cells..."

// Con memoria de largo plazo: se guarda una sola vez
memory.save('design_rule', 'Siempre usa componentes BBVA.');
```

```
// Y el agente la recupera automáticamente en cada ejecución
const rules = await memory.getAll('design_rule');
messages.unshift({ role: 'system', content: rules.join('\n') });
```

IDEA CLAVE

Ya no es necesario repetirlo: la memoria de largo plazo convierte decisiones puntuales en **reglas persistentes** que se aplican a todas las tareas futuras del agente.

EJERCICIO

Guarda 3 reglas de convención en la memoria de largo plazo de tu agente y confirma, generando un componente nuevo sin mencionarlas en el prompt, que igual las respeta.

FP

PROYECTO FINAL

AI Cells Software Engineer

Un sistema de agentes que recibe una historia de usuario y genera automáticamente un componente Lit listo para revisión, integrando todo lo construido en los 17 módulos.

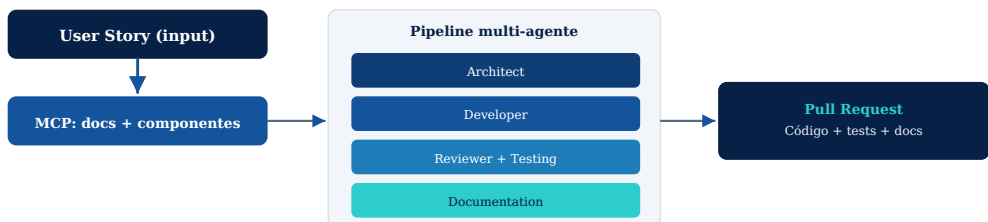
→ ENTRADA

→ "Como cliente quiero visualizar mi saldo disponible para poder consultar mi cuenta."

✓ Salida esperada

- Analiza la historia de usuario (Architect Agent, Módulo 13).
- Diseña la estructura del componente.
- Genera un componente **Lit** siguiendo las convenciones de **Cells** (Módulos 3, 7).
- Crea estilos y soporte para temas.
- Genera pruebas unitarias con **@open-wc/testing** (Módulo 10).
- Produce la documentación — README y API (Módulo 11).
- Genera historias para **Storybook** (Módulo 12).
- Ejecuta una revisión automática del código con recomendaciones de mejora (Módulo 8).
- Produce un resumen de cambios listo para un Pull Request (Módulos 14-15).

El sistema completo, de un vistazo



★ Competencias que desarrollarán los alumnos

Al finalizar el curso, los participantes serán capaces de:

Diseño de agentes

Diseñar agentes de IA orientados al desarrollo de software.

Automatización Lit/Cells

Automatizar la creación de componentes Lit siguiendo la arquitectura Cells.

Integración de LLMs

Integrar LLMs con herramientas de desarrollo mediante MCP y Function Calling.

Calidad asistida por IA

Aplicar IA para revisión de código, refactorización, pruebas y documentación.

Sistemas multi-agente

Construir flujos de trabajo con múltiples agentes especializados.

Integración con plataformas

Integrar agentes con GitHub y Azure DevOps para acelerar el ciclo completo de desarrollo.

CIERRE DEL PROGRAMA

El AI Cells Software Engineer no reemplaza al ingeniero: automatiza el trabajo repetitivo del ciclo de vida de un componente para que el equipo dedique más tiempo a decisiones de arquitectura, experiencia de usuario y revisión crítica.